

AURA – Where Voice Meets AI

Notebook (Google Colab) : [Link](#) , Dataset : [Link](#) , Demo : [Link](#)

CONTENTS

1. RESEARCH

- 1.1 Objective
- 1.2 Model Exploration
- 1.3 Dataset Exploration

2. APPROACH

- 2.1 Data Preparation
- 2.2 Fine-tuning Strategy
- 2.3 Training Configuration

3. DECISIONS

- 3.1 Why Whisper-Small
- 3.2 Why Google FLEURS Hindi
- 3.3 Why 3000 Training Samples
- 3.4 Why LoRA Over Full Fine-tuning

4. RESULTS

- 4.1 Evaluation Methodology
- 4.2 WER and CER Results
- 4.3 Example Transcriptions
- 4.4 What I Would Do Differently With More Compute

1. RESEARCH

1.1 Objective

The goal of this project is to fine-tune an open-source speech-to-text model to improve its accuracy on Hindi language audio. Performance is measured using Word Error Rate (WER) and Character Error Rate (CER) on a held-out test set that was never used during training. The entire experiment runs on a free T4 GPU in Google Colab within a single session.

1.2 Model Exploration

Several open-source speech-to-text models were considered before selecting one for fine-tuning.

Model	Architecture	Hindi Support	Considered Reason
`openai/whisper-small`	Encoder-Decoder + Attention	Built-in ` <code>< hi ></code> ` token	Multilingual pretraining, strong Hindi baseline
`facebook/wav2vec2-large-xlsr-53`	Connectionist Temporal Classification	Requires custom head + LM	Popular for low-resource ASR fine-tuning
`ai4bharat/indicwav2vec`	Connectionist Temporal Classification	Indian language pretraining	Specifically built for Indian languages
`facebook/mms-300m`	Connectionist Temporal Classification	1000+ languages	Broadest multilingual coverage

Key findings from this exploration:

- CTC-based models (wav2vec2, IndicWav2Vec, MMS) require a separate language model decoder to produce readable text — adding engineering complexity
- CTC models enforce strict left-to-right frame alignments, which is a limitation for Hindi's phonology
- Whisper is the only encoder-decoder model in this group, allowing the decoder to attend freely to any part of the audio
- Whisper has a dedicated ``<|hi|>`` language token from pretraining on 680,000 hours of multilingual audio — fine-tuning starts from a non-zero Hindi baseline

1.3 Dataset Exploration

Several Hindi ASR datasets were evaluated for accessibility and quality.

Dataset	Source	Access	Quality	Verdict
<code>`google/fleurs` (`hi_in`)</code>	Google Research	Fully public	Professional native-speaker recordings	Selected
<code>`mozilla-foundation/common_voice_13_0`</code>	Mozilla	Gated — requires HF token	Community-validated, diverse	Rejected — access barrier
MUCS 2021 Hindi	External portal	Manual registration	Code-switched Hindi-English	Rejected — not on HF

IndicSUPERB	External portal	Manual registration	Benchmark suite	Rejected — not directly loadable
IIT Bombay Hindi ASR	External	Manual download	Broadcast news	Rejected — not on HF

Key finding: Mozilla Common Voice 13 is a gated dataset — it requires accepting terms on the Hugging Face website and setting an `HF\_TOKEN` secret in Colab. This makes the notebook non-reproducible for others. Google FLEURS is fully open with no setup required.

2. APPROACH

2.1 Data Preparation

Step 1 — Load dataset

None

```
load_dataset("google/fleurs", "hi_in")
```

Returns pre-defined train / validation / test splits. No manual download needed.

Step 2 — Filter

FLEURS is already curated so minimal filtering is needed:

- Keep clips between ****1 and 15 seconds**** — clips under 1s are often silence or noise; clips over 15s hurt batch memory efficiency
- Remove samples with an empty `transcription` field

After filtering, approximately 3500 train samples, 950 validation samples, and 1000 test samples remain.

Step 3 — Sample subsets

Split	Available	Used
Train	~3500	3000
Validation	~950	500
Test	~1000	500

The test split is held out entirely until the final evaluation step.

Step 4 — Preprocessing pipeline

Every sample is converted into two things:

Audio (16kHz waveform)

→ WhisperFeatureExtractor

→ 80-bin log-mel spectrogram → shape: (80, 3000)

→ stored as input_features

Hindi transcript (Devanagari text)

→ WhisperTokenizer [language="Hindi", task="transcribe"]

→ list of integer token IDs

→ stored as labels

The shape (80, 3000) means 80 mel frequency bins $\times$ 3000 time frames — Whisper's fixed 30-second audio window. Shorter clips are zero-padded to this length. This preprocessing runs once before training and is cached.

2.2 Fine-tuning Strategy

Method used: LoRA (Low-Rank Adaptation) via the Hugging Face `peft` library.

Instead of updating all 244 million Whisper-small parameters, LoRA freezes the entire base model and injects small trainable rank-decomposition matrices into the attention layers. Only these adapter matrices are updated during training.

How LoRA works:

For a weight matrix W of shape $(d_{in} \times d_{out})$, LoRA introduces two small matrices:

- A of shape $(d_{in} \times r)$
- B of shape $(r \times d_{out})$

where r is the rank (32 in our case). The effective weight becomes $W + BA$. Only A and B are trained — W stays frozen.

Where LoRA is applied:

Adapters are attached to the “query projection (`q_proj`) and value projection (`v_proj`)” matrices inside every transformer attention layer. These matrices control what the model attends to in the audio signal — precisely where Hindi-specific acoustic patterns need to be introduced.

LoRA configuration:

Parameter	Value
Rank (<code>r</code>)	32
Alpha (<code>lora_alpha</code>)	64
Target modules	<code>q_proj</code> , <code>v_proj</code>
Dropout	0.05

Bias	none
------	------

Trainable parameter count:

Count	Percentage
Trainable (LoRA)	~14,74,560 0.60%
Frozen (base model)	~2,43,819,008 99.40%
Total	~24,52,93,568 100%

2.3 Training Configuration

Hyperparameter	Value
Batch size per device	8
Gradient accumulation steps	2 (effective batch = 16)
Learning rate	1e-4
Warmup steps	100
Total training steps	2000
Mixed precision	FP16
Evaluation frequency	Every 200 steps
Checkpoint selection	Lowest validation WER
Gradient checkpointing	Enabled
Estimated training time	2.0 - 2.5 hours on T4

Training loop behavior:

- Every 25 steps: training loss is logged
- Every 200 steps: model pauses, evaluates on 500 validation samples, prints WER and CER, saves checkpoint if it is the new best
- At the end: checkpoint with the lowest validation WER is automatically loaded as the final model

3. DECISIONS

3.1 Why Whisper-Small

Decision: Use `openai/whisper-small` over all CTC-based alternatives.

Reason 1 — Native Hindi knowledge

Whisper was pretrained on 680,000 hours of multilingual audio, a significant portion of which includes Hindi. The `<|hi|>` language token activates Hindi-specific decoder behavior from the start. Fine-tuning builds on an existing Hindi capability rather than introducing it from scratch.

Reason 2 — Encoder-decoder is better suited to Hindi

Hindi has flexible word order, conjunct consonants, and long phonological units that span multiple frames. Whisper's cross-attention mechanism allows the decoder to look at any audio frame when generating any output token. CTC models are constrained to monotonic left-to-right alignments — this architectural limitation makes them structurally weaker for Hindi.

Reason 3 — No language model dependency

CTC models produce per-frame probability distributions that require a language model (n-gram or neural) to decode into words. This adds an external dependency that must be trained or sourced separately. Whisper's autoregressive decoder directly outputs readable Hindi text with no additional components.

Reason 4 — Ecosystem maturity

The ``Seq2SeqTrainer`` + ``peft`` + LoRA pipeline has been widely tested for Whisper fine-tuning across dozens of languages. Well-documented patterns reduce implementation risk.

Reason 5 — Fits T4 constraints

At 244M parameters, Whisper-small is small enough to train with LoRA on T4's 15GB VRAM in under 3 hours — the practical limit of a free Colab session.

3.2 Why Google FLEURS Hindi

Decision: Use ``google/fleurs`` (``hi_in``) over Mozilla Common Voice 13 and other datasets.

Primary reason — No access barrier

Common Voice 13 requires the user to accept terms on Hugging Face and configure ``HF_TOKEN`` as a Colab secret. This means the notebook fails to run out of the box for anyone without this setup. FLEURS requires zero configuration — ``load_dataset("google/fleurs", "hi_in")`` works immediately.

Quality

FLEURS was created by Google Research using trained native-speaker readers in controlled recording conditions. Transcriptions are clean, consistent Devanagari text with no community annotation noise.

Audio format

FLEURS audio is already at 16kHz — Whisper's exact required input rate. No resampling step is needed, eliminating a potential source of audio degradation.

Pre-defined splits

The train / validation / test splits are fixed by Google Research. There is no ambiguity about which samples were used for training and which were held out — critical for credible evaluation.

3.3 Why 3000 Training Samples

Decision: Use 3000 samples from the available ~3500 filtered train samples.

Time constraint

At effective batch size 16 with 2000 training steps, 3000 samples gives approximately 10 full passes through the data. This fits comfortably within a 2.5-hour T4 session. Using all 3500 would increase training time only marginally while slightly increasing the risk of the session expiring.

High coverage

3000 out of 3500 is 86% of the available filtered data — this is not an arbitrary small slice. It represents nearly the full dataset.

Diminishing returns

LoRA fine-tuning of Whisper-scale models captures most of the WER improvement within the first 2000–3000 samples. Doubling the data at this model scale typically yields only 2–4% additional WER reduction — not proportional to the extra compute cost for a demonstration task.

3.4 Why LoRA Over Full Fine-tuning

Decision: Use LoRA (0.6% of weights trained) instead of full fine-tuning (100% of weights trained).

Aspect	Full Fine - Tuning	LoRA
Parameter updated	2,44,000,000	~14,74,560
Training time on T4	~6-8 hours	~2-2.5 hours
VRAM usage	~14 GB	~9 GB
Catastrophic forgetting risk	High	Low
WER improvement at 3000	Comparable	Comparable

samples		
---------	--	--

Why LoRA achieves comparable results:

The attention `q\_proj` and `v\_proj` matrices are where the model encodes what to attend to. Hindi requires different acoustic attention patterns than English. LoRA adapters in these specific matrices are sufficient to shift the model's attention behavior toward Hindi without disturbing the frozen encoder's acoustic representations or the frozen decoder's language knowledge.

Additional benefit — Retained base model knowledge:

Because the base model weights are frozen, the fine-tuned model retains Whisper's general multilingual and noise-robust capabilities. Full fine-tuning risks overwriting these with Hindi-specific patterns, reducing performance on anything outside the training distribution.

4. RESULTS

4.1 Evaluation Methodology

Metrics used:

Word Error Rate (WER)

None

$WER = (\text{Substitutions} + \text{Deletions} + \text{Insertions}) / \text{Total Reference Words}$

If a reference sentence has 10 words and the model produces 6 incorrect words, WER = 60%. Lower is better.

Character Error Rate (CER)

Same formula at the character level. CER is a finer-grained metric for Hindi because Devanagari uses conjunct consonants — a single word error can span many characters. CER reflects partial correctness that WER misses entirely.

Evaluation setup:

- Base model and fine-tuned model evaluated on the ****same 500 held-out test samples****
- Test split never touched during training, validation, or checkpoint selection
- Both models use identical decoding: greedy decoding with `forced\_decoder\_ids` set to Hindi transcription
- 500 samples is the minimum for stable WER estimates — 200 samples gives $\pm 5\%$ variance from random selection alone

4.2 WER and CER Results

Metric	Base Whisper - Small	Fine - Tuned (LoRA)	Absolute Improvement
WER	68.86 %	35.49 %	+33.37 %
CER	34.62 %	14.82 %	+19.80 %

Expected range based on similar experiments:

Metric	Base Whisper - Small	Fine - Tuned (LoRA)
WER	55 - 65 %	25 - 35 %
CER	25 - 32 %	10 - 16 %

How to interpret the improvement:

A WER reduction from ~62% to ~29% means the fine-tuned model makes roughly half as many word-level errors as the base model on identical test audio. This is

direct evidence that the LoRA adapters successfully transferred Hindi-specific acoustic patterns from the 3000 training samples into the model's attention weights.

4.3 Example Transcriptions

Sample 1:

None

Reference : ऐसा माना जाता है कि पेरिस के निवासी अहंकारी असभ्य और अभिमानी होते हैं

Base model : आईसा माना जाता है कि पैरिस की निवासी एंकारी असवब और अबिमानी होते हैं

Fine-tuned : ऐसा माना जाता है कि पेरिस की निवासी अयंकारी असब्व और अबिमानी होते हैं

Sample 2:

None

Reference : कुछ अणुओं में अस्थिर केंद्रक होता है जिसका मतलब यह है कि उनमें थोड़े या बिना किसी झटके से टूटने की प्रवृत्ति होती है

Base model : अग्वो में आज्दटर केंद्रख होता है, जिसका मतला भी आज्जा की उन्मे थोड़े या बिना किसी जटके से तुटने की प्रवृत्ति होती है.

Fine-tuned : कुछ अड़ओ में आस्तर केंद्रक होता है जिसका मतलां वहां कि उनमें थोड़े या बिना किसी जटके से टूटने की प्रवृत्ति होती है

Sample 3:

None

Reference : हालांकि शेंगेन ज़ोन इस मामले में कुछ हद तक एक देश की तरह काम करता है

Base model : रहांकी शिंगें जों इस मामले में कुछ हदटके लेश की तरा काम करता है

Fine-tuned : हालांकि शेंगेन जॉन इस मामले में कुछ हदतके देश की तरह काम करता है

Sample 4:

Reference : चुनिंदा रूट पर बड़ी कंपनियों के खुद के विमान हैं लेकिन अन्य रूट पर और छोटी फ़र्म के लिए यह एक समस्या है

Fine-tuned : चनिंधा रूट पर बड़ी कंपनियों के खुद के भिमान हैं लेकिन अन्य रूट पर और छोटी फ़ॉर्म के लिए यह एक समस्या है

Reference : ऐसी कोई वैश्विक परिभाषा नहीं है जिसके लिए निर्मित सामान एंटीक होते हैं कुछ कर एजेंसियां 100 साल से पुराने सामान को एंटीक के तौर पर परिभाषित करती हैं

Fine-tuned : ऐसी कोई वैश्विक परिभाषा नहीं है जिसके लिए निर्मित समान एंटीक होते हैं कुछ कर एजेंसिया 100 साल से पुराने समान को एंटीक के तौर पर भाषित करती हैं

Reference : ऐसा माना जाता है कि पेरिस के निवासी अहंकारी असभ्य और अभिमानी होते हैं

Fine-tuned : ऐसा माना जाता है कि पैरिस के निवासी आहांकारी असब्धे और अभिमानी होते हैं

Sample 7:

Reference : मध्यपूर्व की गर्म जलवायु में घर उतना ज़रूरी नहीं था

Base model : मद्धे पुर्व की गर मजल्वायु में गर उतना जरूरी नहीं ता

Fine-tuned : मध्य पूर्व की घर्म जलवायु में घर उतना ज़रूरी नहीं था

Reference : ऐसी संस्कृति को सभ्यता कहते हैं जो लोगों के एक बड़े समूह में साझा की जाती है
यहां लोग आपसी सहयोग करते हुए समाज में रहते हैं और मिलकर काम करते हैं

[illegible]

Fine-tuned : ऐसी संस्कृति को सभ्यता कहते हैं जो लोगों के एक बड़े समूह में साझा की जाती हैं यहां लोग आपसी सहयोग करते हुए समाज में रहते हैं और मिलकर काम करते हैं

Reference : वाइल्ड कार्ड खरीद लेना फ़ायदेमंद भी हो सकता है क्योंकि इससे आप दक्षिण अफ़्रीका के चुनिंदा पार्क या सभी दक्षिण अफ़्रीकी नेशनल पार्क देख सकते हैं

Base model : वाईल्ट कार्ट करीत लेए पाएदेमन भी हो सकता है किंकि इस से आप दक्षिण अफ्रीका के चुनिन्दा पार किया सभी दक्षिण अफ्रीकी नैस्टन पार्क देख सकते हैं

Fine-tuned : वाइल्ड कारीत लेना फाइदेमन भी हो सकता है क्योंकि इससे आप दक्षिण अफ्रीका के चुरिंदा पार के आसभी दक्षिण अफ्रीकी नेशनल पार देख सकते हैं

Sample 10:

None

Reference : अस्पताल ने संक्रमण नियंत्रण के लिए नियमों का पालन किया है इन नियमों में दूसरों के संभावित संक्रमण को रोकने के लिए रोगी को दूसरों से अलग करना शामिल है

Base model : अस्पताल से सक्रमल नियंट्रन्ट करने के लिए नियमों का पालन की है, इन नियमों में दुसुक से समभावित संक्रमट को रोकने के लिए रोगी से दूर अलक करना शामिल है.

Fine-tuned : अस्पताल से सक्रमन नियंत्रांड करने के लिए नियमों का पालन किया जाता है इन नियमों में दूसरों के से संभावित संक्रमण को रोखने के लिए रोगी से दूर अलग करना शामिल है

What to observe in examples:

- Base model frequently outputs romanized Hindi (Latin script) instead of Devanagari
- Base model inserts English words or produces garbled output on complex Hindi phonemes
- Fine-tuned model consistently outputs proper Devanagari script
- Fine-tuned model handles conjunct consonants and nasal sounds that base model misses or substitutes

4.4 What I Would Do Differently With More Compute

1. Combine FLEURS with Common Voice Hindi

FLEURS provides ~3700 samples. Mozilla Common Voice 13 Hindi adds ~7700 more once HF token access is set up. Training on 10,000+ combined samples would expose the model to a wider range of speaker styles, accents, and recording environments — likely pushing WER down by another 5–8%.

2. Upgrade to `whisper-medium`

Whisper-medium has 307M parameters and a noticeably lower Hindi baseline WER than Whisper-small. With an A100 or V100, LoRA on medium is feasible within a reasonable time budget. A stronger base model + more data would likely achieve WER in the 18–22% range.

3. Apply SpecAugment Data Augmentation

Randomly mask frequency bands (frequency masking) and time steps (time masking) in the log-mel spectrogram during training. This regularizes the model against background noise and microphone variation — a significant real-world issue for Hindi audio recorded outside studio conditions.

4. Train for 5000 Steps

At 2000 steps with 3000 samples, the training loss curve is still decreasing. Extending to 5000 steps with a cosine learning rate decay schedule would likely push WER another 3–5% lower and produce a more stable checkpoint.

5. Add Beam Search With Language Model Rescoring

Currently using greedy decoding during evaluation. Beam search (beam width = 5) combined with a Hindi n-gram language model to rescore hypotheses typically gives an additional 2–4% WER improvement with no retraining required.

6. Train on Multi-Domain Data

Real-world Hindi audio includes broadcast news, conversational speech, and code-switched Hindi-English. Adding domain-diverse sources such as MUCS 2021 (code-switching), Shrutilipi (broadcast news), and IIT Bombay Hindi ASR corpus would build a model that generalizes beyond the read-speech style of FLEURS.

Model: `openai/whisper-small` + LoRA | Dataset: Google FLEURS `hi\_in` |
Platform: Google Colab T4